Lecture 13: Neural Network Optimization

Tao Lin

SoE, Westlake University

May 12, 2025

Table of Contents

1. Part 1: Advanced Back-propagation & Gradient Computation
2. Part 2: Neural Network Optimizers
3. Part 3: Supervised Learning Paradigm & Advanced Applications

Neural Network Optimization - Part 1: Advanced Back-propagation

- Overview:
  - Understanding back-propagation as the engine for gradient computation in NNs.
  - How gradients are calculated efficiently using reverse mode automatic differentiation.
  - The role of Vector-Jacobean Products (VJPs) for common neural network layers.
  - Back-propagation through general computation graphs.

Recap: What is Back-propagation?

- Back-propagation is an algorithm for efficiently computing gradients in a neural network (or any computation graph).
- It applies the chain rule of calculus recursively.

Neural Network Optimization - Part 2: Optimizer Algorithms

- This section delves into various algorithms used to optimize neural network parameters, building upon the gradient computation methods discussed previously.

The Optimization Problem in Deep Learning

- **Goal:** Minimize a loss function $L(\boldsymbol{\theta})$ with respect to the model parameters $\boldsymbol{\theta}$.

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} L(\boldsymbol{\theta})$$

- In deep learning, $L(\boldsymbol{\theta})$ is typically:
  - **Non-convex:** Many local minima and saddle points. Global optimum is not guaranteed.
  - **High-dimensional:** $\boldsymbol{\theta}$ can contain millions or billions of parameters.
- A common loss is the Negative Log-Likelihood (NLL) for supervised learning:

$$\mathcal{L}(\boldsymbol{\theta}) = -\log p(\mathcal{D}|\boldsymbol{\theta}) = -\sum_{n=1}^{N} \log p(y_n|x_n;\boldsymbol{\theta})$$

  (Equation 13.68 from @ch13-opt)
- It is also common to add a regularizer to the loss.

Gradients: The Compass for Optimization

- Gradients $\nabla_{\boldsymbol{\theta}} L$ indicate the direction of steepest ascent of the loss function.
- Optimizers use $-\nabla_{\boldsymbol{\theta}} L$ to iteratively update parameters $\boldsymbol{\theta}$ towards a minimum.
- Gradients are computed efficiently via **back-propagation** (as covered in Part 1 / Lecture 13.3).
- Standard approach: Use back-prop to compute gradients and pass them to an off-the-shelf optimizer.

Challenges in DNN Optimization (1/2)

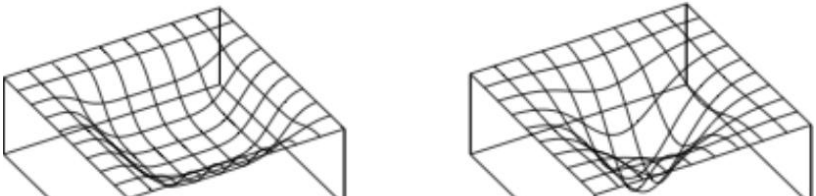
- **Learning Rate Sensitivity:**
  - The learning rate (η) is a crucial hyper-parameter.
  - Too high: Overshooting, divergence. Too low: Very slow convergence.
  - Learning rate schedules (dynamic adjustment of η) are often employed. (See Section 13.4.1 @ch13-opt)
- **Vanishing and Exploding Gradients:** (See Section 13.4.2 @ch13-opt)
  - In deep networks, gradients are back-propagated through many layers.
  - The gradient signal can diminish (vanish) or grow excessively (explode).
  - Gradient of loss w.r.t. activations $\mathbf{z}_l$ at layer l:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_{l+1}} \frac{\partial \mathbf{z}_{l+1}}{\partial \mathbf{z}_l} = \mathbf{J}_l \mathbf{g}_{l+1}$$

    (Equation 13.69 from @ch13-opt, where $\mathbf{J}_l$ is Jacobean, $\mathbf{g}_{l+1}$ is gradient from next layer)
  - Behavior depends on spectral radius of $\mathbf{J}_l$.

Challenges in DNN Optimization (2/2)

- **Vanishing/Exploding Gradients (Cont.):**
  - **Exploding Gradients:** Can be mitigated by **gradient clipping**: $\mathbf{g}' = \min\left(1, \frac{c}{|\mathbf{g}|}\right)\mathbf{g}$ (Equation 13.70 from @ch13-opt, where c is a threshold)
  - **Vanishing Gradients:** More complex. Solutions include careful initialization, non-saturating activation functions (e.g., ReLU), and architectural changes (e.g., ResNets, LSTMs).
- **Saddle Points & Plateaus:**
  - Regions where $\nabla_{\boldsymbol{\theta}} L \approx \mathbf{0}$ but are not (local) minima.
  - More common than local minima in high-dimensional spaces, can significantly slow convergence.
- **Poor Conditioning:**
  - Hessian matrix $H = \nabla^2_{\boldsymbol{\theta}} L$ has a wide range of eigenvalues.
  - Loss surface is much steeper in some directions than others (like a long narrow ravine).
  - Causes slow progress for simple gradient descent.

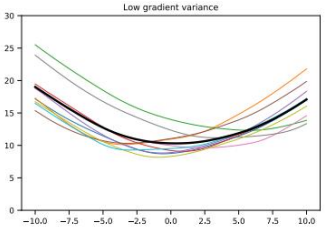


Stochastic Gradient Descent (SGD)

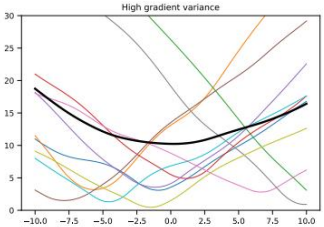
- **Update Rule:** Parameters are updated using gradients computed on a mini-batch of data:

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} L_{batch}(\boldsymbol{\theta}_t)$$

  where η is the learning rate, and L_{batch} is the loss on the current mini-batch.
- **Noise from Mini-batches:**
  - The stochasticity (noise) introduced by mini-batch sampling can be beneficial.
  - Helps escape sharp local minima and saddle points.
  - Can act as an implicit form of regularization.



(a)



(b)

Figure: SGD implicitly prefers minima that are stable across different mini-batches, contributing to its

SGD: Advantages and Disadvantages

- **Pros:**
  - Simple to implement and understand.
  - Computationally cheap per update (scales with batch size, not dataset size).
  - Can generalize well, often finding flatter minima.
  - Theoretical convergence guarantees under certain conditions (though for non-convex, it's to critical points).
- **Cons:**
  - High variance in parameter updates can lead to noisy convergence.
  - Can be slow to converge, especially if the loss landscape has ravines or is poorly conditioned.
  - Highly sensitive to the choice of learning rate η and initialization.
  - May get stuck in suboptimal local minima or oscillate around a minimum.

SGD with Momentum

- **Intuition:** Accelerate SGD in consistent directions of descent and dampen oscillations. Like a heavy ball rolling down a hill, accumulating momentum.
- **Update Rules:**
  - Velocity ($\mathbf{v}_t$) accumulates an exponentially decaying moving average of past gradients:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1-\beta)\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t)$$

    (Note: Another common form for velocity is $\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t)$, where η is applied later)
  - Parameters are updated using this velocity:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{v}_t$$

    (If using the alternative velocity form above, the parameter update might be $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \mathbf{v}_t$ if η is incorporated into $\mathbf{v}_t$, or $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{v}_t$).
- β is the momentum parameter (e.g., common values are 0.9, 0.99).

SGD with Momentum: Benefits

- **Accelerated Convergence:**
  - Speeds up convergence, especially along directions of low curvature (ravines) where SGD would oscillate.
  - Consistent gradients add up, while oscillating components tend to cancel out.
- **Improved Optimization:**
  - Helps to navigate over small local minima and saddle points more effectively.
- **Reduced Oscillations:**
  - Dampens oscillations often seen with plain SGD in steep ravines.
- Still requires careful tuning of learning rate η and momentum β.

Adaptive Learning Rate Methods: Overview

- **Motivation:**
  - Different parameters in a model might require different learning rates.
  - Some parameters might be updated frequently (e.g., for common features), others infrequently.
  - The optimal learning rate might change during training.
- **Idea:** Automatically adapt the learning rate for each parameter (or group of parameters) based on the history of its gradients.
- **Goal:** Achieve faster convergence, robustness to initial learning rate choice, and handle sparse gradients effectively.

AdaGrad: Adaptive Gradient Algorithm

- **Intuition:** Assign smaller learning rates to parameters with historically large or frequent gradients, and larger learning rates to parameters with small or infrequent gradients.
- **Mechanism:**
  - Accumulates the sum of squared gradients for each parameter θ_i:

$$G_{t,ii} = G_{t-1,ii} + (\nabla_{\theta_i} L(\boldsymbol{\theta}_t))^2$$

    (where G_t is typically a diagonal matrix, or $G_{t,ii}$ is a per-parameter accumulator. The square is element-wise.)
  - **Update Rule (per parameter θ_i):**

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} \nabla_{\theta_i} L(\boldsymbol{\theta}_t)$$

    where η is a global learning rate and ϵ is a small constant for numerical stability (e.g., 10^{-8}).

AdaGrad: Advantages and Disadvantages

- **Pros:**
  - Well-suited for problems with sparse gradients (e.g., in Natural Language Processing for word embeddings, where some words are rare).
  - Eliminates the need to manually tune the learning rate as aggressively for each parameter (though the global η still needs setting).
- **Cons:**
  - **Aggressively Decaying Learning Rate:** The accumulated sum of squares $G_{t,ii}$ in the denominator grows monotonically throughout training.
  - This causes the effective learning rate to decrease continuously and eventually become infinitesimally small, potentially stopping learning prematurely before reaching an optimal solution.

RMSProp: Addressing AdaGradś Limitation

- **Motivation:** To resolve AdaGrad's problem of an aggressively and monotonically decreasing learning rate.
- **Mechanism:**
  - Uses an exponentially decaying average of squared gradients instead of accumulating all past squared gradients.
  - Per-parameter squared gradient average $E[g^2]_{t,i}$:

$$E[g^2]_{t,i} = \gamma E[g^2]_{t-1,i} + (1-\gamma)(\nabla_{\theta_i} L(\boldsymbol{\theta}_t))^2$$

    (The square is element-wise.)
  - **Update Rule (per parameter θ_i):**

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{E[g^2]_{t,i} + \epsilon}} \nabla_{\theta_i} L(\boldsymbol{\theta}_t)$$

- γ is a decay rate (hyper-parameter, e.g., 0.9), η is a global learning rate, ϵ for stability.
- This prevents the denominator from growing unboundedly.

Adam: Adaptive Moment Estimation (1/2)

- **Combines Ideas:** Integrates the benefits of Momentum (using first moment of gradients) and RMSProp (using second moment of gradients for adaptive learning rates).
- **Moment Estimates:**
  - **First Moment (Mean of Gradients):** $\mathbf{m}_t$ (like momentum, stores an exponentially decaying average of past gradients)

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t)$$

  - **Second Moment (Uncentered Variance of Gradients):** $\mathbf{v}_t$ (like RMSProp, stores an exponentially decaying average of past squared gradients)

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)(\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t))^2$$

  (The square is element-wise.)
- β_1 and β_2 are exponential decay rates (hyper-parameters, e.g., $\beta_1 = 0.9, \beta_2 = 0.999$).

Adam: Adaptive Moment Estimation (2/2)

- **Bias Correction:** $\mathbf{m}_t$ and $\mathbf{v}_t$ are initialized as zeros and are biased towards zero, especially during initial training steps. Adam corrects for this bias:
  - Corrected first moment: $\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}$
  - Corrected second moment: $\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t}$

  (where t is the time-step)
- **Update Rule:**

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}\hat{\mathbf{m}}_t$$

  (Operations $\sqrt{\cdot}$ and division are element-wise.)
- **Popularity:** Adam is often a good default optimizer due to its robust performance across a variety of tasks and architectures. (Mentioned in @ch13-opt as a popular choice).

AdamW: Decoupled Weight Decay

- **Problem with L2 Regularization in Adam:** When L2 regularization (weight decay) is added to the loss function, its gradient term $\lambda \boldsymbol{\theta}_t$ gets scaled by the adaptive learning rates $(1/\sqrt{\hat{\mathbf{v}}_t + \epsilon})$ in Adam. This can make weight decay less effective for parameters with large historical gradient magnitudes.
- **AdamW Solution:** Decouples weight decay from the gradient-based update.
  1. Compute Adam's parameter update (moment-based part):
  $$\Delta \boldsymbol{\theta}_t^{\text{Adam}} = \frac{\eta}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \hat{\mathbf{m}}_t$$
  2. Update parameters using Adam's step:
  $$\boldsymbol{\theta}_{t+1}^{\text{intermediate}} = \boldsymbol{\theta}_t - \Delta \boldsymbol{\theta}_t^{\text{Adam}}$$
  3. Apply weight decay directly to the weights:
  $$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_{t+1}^{\text{intermediate}} - \eta \lambda \boldsymbol{\theta}_t$$

  (Here, λ is the weight decay factor, and $\boldsymbol{\theta}_t$ refers to the parameters *before* the current step's Adam update. The learning rate η might be the main LR or a separate scheduled WD rate.)
- **Benefit:** Often leads to better generalization and training performance compared to standard Adam with L2 regularization in the loss.

Second-Order Optimization Methods

- **Idea:** Utilize second-derivative information (the Hessian matrix $H = \nabla^2_{\boldsymbol{\theta}} L$) to get a better sense of the loss surface curvature.
- **Newton's Method Update:**

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - H_t^{-1} \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t)$$

- **Examples:** Newton's method, Quasi-Newton methods (e.g., L-BFGS which approximates H_t^{-1}).
- **Pros:** Can converge very rapidly (quadratically) near a good minimum.
- **Cons for Deep Learning:**
  - **Computationally Prohibitive:**
    - Computing the Hessian: $O(N_{params}^2)$ operations.
    - Storing the Hessian: $O(N_{params}^2)$ memory.
    - Inverting the Hessian: $O(N_{params}^3)$ operations.
  - Impractical for typical DNNs with millions/billions of parameters (N_{params}).

Optimizer Comparison and Practical Choices

- **No Single Best Optimizer:** The performance of an optimizer can vary significantly depending on the specific problem, model architecture, dataset, and hyper-parameter tuning.
- **Common Starting Points:**
  - **Adam / AdamW:** Often good default choices. They are generally robust, converge relatively fast, and work well across a wide range of problems. (Adam mentioned as popular in @ch13-opt)
  - Typical Adam hyper-parameters: $\eta = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$.
- **SGD with Momentum:**
  - Can sometimes achieve better generalization performance than adaptive methods, especially with careful and extensive tuning of learning rate schedules and momentum.
  - May require more effort to tune and might converge slower initially.
- **Experimentation is Key:** It's often beneficial to try a few different optimizers and learning rate schedules.

Learning Rate Schedules: Guiding the Descent

- **Motivation:** The optimal learning rate often changes during training. Starting large can help escape poor local minima, while decreasing it later can help converge to a good solution.
- **Common Schedules:**
  - **Step Decay:** Reduce η by a factor (e.g., 0.1) at predefined epochs.
  - **Exponential Decay:** $\eta_t = \eta_0 e^{-kt}$.
  - **Cosine Annealing:** $\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\frac{t_{curr}}{T_{total}}\pi))$. Smoothly varies η from η_{max} to η_{min}.
  - **Linear Decay:** Decrease η linearly over epochs.
  - **Warmup:** Start with a very small η for a few initial epochs and gradually increase it to the target η. Helps stabilize training in the early phase, especially for large models or batches.
- Learning rate schedules are commonly used in conjunction with all types of optimizers.

Broader Context: Training Aids and Summary

- Optimizers are a critical component, but successful deep learning training relies on a combination of factors:
  - **Initialization:** Proper weight initialization (e.g., Xavier/Glory, He) is crucial to prevent early vanishing/exploding gradients.
  - **Regularization:** Techniques to prevent overfitting and improve generalization:
    - L2 Regularization (Weight Decay)
    - Dropout
    - Early Stopping
  - **Batch Normalization:**
    - Standardizes inputs to layers, stabilizing training.
    - Smooths the loss landscape.
    - Allows for higher learning rates.
    - Can act as a regularizer.
- **Summary:** Choosing and tuning an optimizer is an art and science. It involves understanding the trade-offs and often requires empirical validation. The interplay between the optimizer, learning rate schedule, regularization, and model architecture determines the final performance.

The Supervised Learning Paradigm with NNs

- **Goal:** Learn a complex mapping $f : X \to Y$ from (x_i, y_i) pairs.
- NNs as powerful function approximations; optimization finds θ to minimize $L(y, f(x; \theta))$.

Image Classification (Recap)

- **Input:** Image.
- **Output:** Single class label.
- **Loss:** Cross-Entropy.
- Foundation for many computer vision tasks.

Image Tagging / Multi-Label Classification: Definition

- **Input:** Image.
- **Output:** Set of relevant tags (e.g., "sunset", "beach").
- **Task Setup:** Each tag treated as binary classification (present/absent).

Image Tagging / Multi-Label Classification: Architecture & Challenges

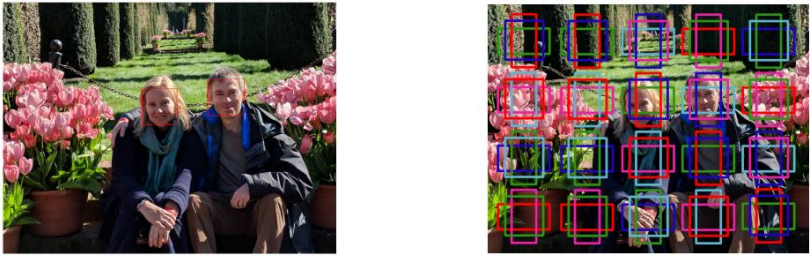
- **NN Architecture:** CNN backbone, output layer with sigmoid per tag.
- **Loss:** Sum of Binary Cross-Entropy losses over tags.
- **Challenges:** Noisy/sparse labels. User-generated tags can be noisy but useful for pre-training.

Object Detection: Introduction

- **Input:** Image.
- **Output:** List of (bounding box, class label) per object.

Object Detection: Anchor Boxes & Prediction

- **Task Setup:** Models often use **anchor boxes** (predefined candidate boxes).
- For each anchor, predict:
  1. Objectness score.
  2. Class probabilities.
  3. Bounding box refinements (offsets).



(a) (b)

Figure: (a) Face detection example. (b) Anchor boxes illustration.

Object Detection: Single-Stage Architectures

- **SSD (Single Shot Detector):** Single feed-forward net predicts boxes/classes directly from feature maps at multiple scales.
- **YOLO (You Only Look Once):** Divides image into a grid; each grid cell predicts bounding boxes and class probabilities. Known for speed.

Object Detection: Two-Stage Architectures

- **Faster R-CNN:**
  - **Stage 1 (Region Proposal Network - RPN):** Proposes candidate object regions.
  - **Stage 2:** Extracts features for these proposals and performs classification and bounding box regression.
- Generally more accurate but can be slower than single-stage detectors.

Instance Segmentation: Introduction

- **Input:** Image.
- **Output:** For *each object instance*, a class label and a pixel-level mask.
- **Task Setup:** Differentiates between objects of the same class (e.g., car1, car2).

Instance Segmentation: Mask R-CNN

- **NN Architecture Example: Mask R-CNN**
  - Extends Faster R-CNN.
  - Adds a branch in parallel with the classification and bounding box regression head to predict a binary segmentation mask for each Region of Interest (RoI).

Semantic Segmentation: Introduction

- **Input:** Image.
- **Output:** A class label for *each pixel* (e.g., "road", "sky", "building").
- **Task Setup:** Does not distinguish between instances of the same class (e.g., all cars get labeled "car").

Semantic Segmentation: Encoder-Decoder Architectures

- **NN Architecture: Encoder-Decoder architectures** are common.

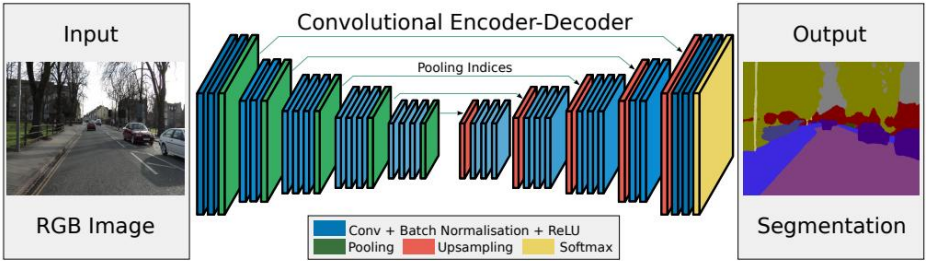


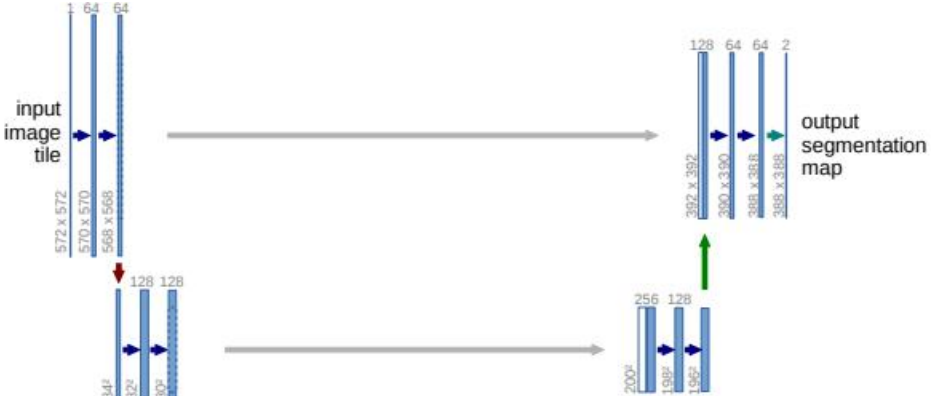
Figure: Figure 14.29: Illustration of an encoder-decoder (aka U-Net style) CNN for semantic segmentation.

Semantic Segmentation: Encoder & Decoder Details

- **Encoder:**
  - Typically a convolutional neural network (e.g., ResNet, VGG) without its final classification layers.
  - Progressively downsamples the image, extracting increasingly complex and abstract features.
  - Captures semantic context.
- **Decoder:**
  - Takes the low-resolution, high-level feature maps from the encoder.
  - Progressively upsamples these features to the original image resolution.
  - Aims to produce pixel-wise class predictions.
  - Common upsampling layers: Transposed Convolutions (sometimes called deconvolution), bilinear upsampling + convolution.

Semantic Segmentation: Skip Connections & U-Net

- **Skip Connections:**
  - Crucial for feeding fine-grained, high-resolution features from the encoder directly to corresponding layers in the decoder.
  - Help the decoder recover spatial detail lost during downsampling in the encoder.
  - Example: **U-Net** architecture is famous for its symmetric encoder-decoder structure with prominent skip connections.



Other Dense Prediction Tasks (Image-to-Image)

- Many computer vision tasks can be framed as **dense prediction**: predicting a value (or vector) for every pixel in an input image.
- These often use similar encoder-decoder structures as semantic segmentation.

Dense Prediction Example: Depth Prediction

- **Task:** Predict the depth (distance from camera) for each pixel in an image.
- **Input:** Single RGB image (monocular depth estimation) or stereo pair.
- **Output:** A depth map (a grayscale image where intensity encodes depth).
- **Loss:** Typically a regression loss (e.g., L1, L2, or scale-invariant losses) on predicted depth values.
- **Applications:** 3D reconstruction, autonomous navigation, augmented reality.

Dense Prediction Example: Surface Normal Prediction

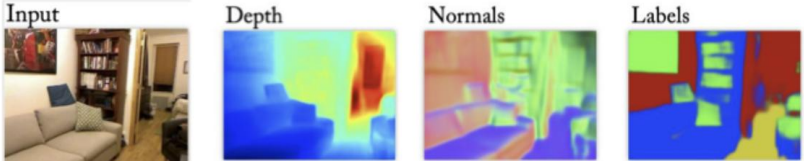
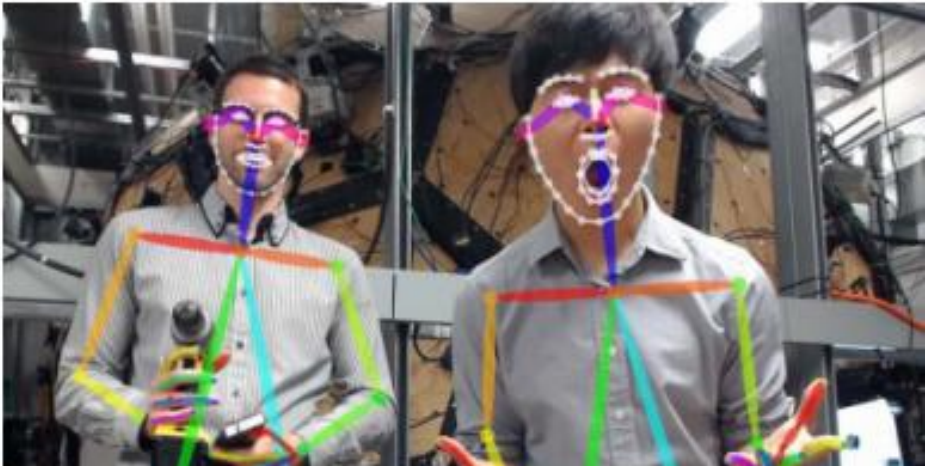
- **Task:** Predict the 3D surface normal vector for each pixel.
- **Input:** Image.
- **Output:** A map where each "pixel" contains (nx, ny, nz) components of the surface normal.
- **Applications:** 3D shape understanding, relighting.
- 

Figure: Figure 14.31: Illustration of a multi-task dense prediction problem, potentially including depth, normals, etc.

Human Pose Estimation: Introduction

- **Input:** Image or video frame.
- **Output:** Locations of skeletal key points (e.g., joints like shoulders, elbows, wrists, hips, knees, ankles).



Human Pose Estimation: Methods & Architecture

- **Task Setup Approaches:**
  - **Heat map-based:** Predict a 2D heat map for each key point type; the peak in the heat map indicates the key point's location. This is a common and robust approach.
  - **Regression-based:** Directly regress the (x,y) coordinates of key points. Can be more challenging.
- **NN Architecture:** Typically a CNN backbone (e.g., ResNet) followed by specialized heads to predict either heat maps or coordinates.
- **Examples:** OpenPose, PersonLab, HRNet.

Concluding on the Supervised Learning Paradigm

- Optimized Neural Networks, driven by back-propagation and powerful optimizers (like Adam, SGD with Momentum), excel at learning complex input-output mappings across a diverse range of tasks.
- The availability of large, high-quality labeled datasets has been a key enabler for the success of supervised learning in deep learning.
- Architectural innovations (e.g., CNNs, Transformers, U-Nets) tailored to specific data types and tasks further enhance performance.